Basi numeriche, TRIS in esadecimale e la luce che scorre

Lezione 5 — 3MAK, SST-INF-01c — Un nuovo progetto da capo, decimale/ottale/binario/esadecimale, accendere e far scorrere i LED con un array

29/10/2026

Nota

Oggi rifacciamo **da capo**, un po' più velocemente, un progetto in **MPLAB X** (utile a chi era assente la volta scorsa e come ripasso per tutti), poi impostiamo i registri **TRIS** della scheda simulata scrivendo i numeri in **esadecimale**, e ripassiamo **come si conta** in basi diverse. Infine facciamo **lampeggiare** i LED e costruiamo una **luce che scorre** sulla porta B, prima nel modo più "faticoso" e poi con un **array** e un ciclo `for`.

Attenzione

Verifica di firmware: giovedì 5 novembre. La verifica riguarda **tutto il contenuto delle dispense da A01 ad A05** (questa compresa); la dispensa sull'installazione di MPLAB X pubblicata su Classroom non è materia di domande. Ci sarà anche un **lavoro da fare a casa**, e quello che farete a casa **entrerà nella valutazione** della verifica: deve essere **scritto da voi**. Copiare il codice prodotto da ChatGPT o da un'altra intelligenza artificiale **equivale a un copia e incolla**; meglio un errore vostro che un programma che non sapete spiegare.

1 Un nuovo progetto in MPLAB X

1. **File** → **New Project** → **Microchip Embedded** → **Application Project**, poi *Next*.
2. Come dispositivo scegliete **PIC16F628A**, il microcontrollore che abbiamo scelto. Il suo **datasheet** (200–300 pagine) contiene tutte le caratteristiche della macchina: vi invito a studiarlo. Per ora usiamo il **simulatore** per concentrarci sul firmware; quando avremo il circuito vero, lo guarderemo.
3. Nella parte di debug non c'è niente da scegliere: *Next*.
4. Come compilatore scegliete **XC8**. Se sul vostro computer c'è la versione 3.0 invece della 3.10, **va bene lo stesso**: non fissatevi sul numero di versione.
5. Scegliete la cartella e il **nome del progetto**. Evitate per quanto possibile **trattini, caratteri speciali e spazi**: a volte creano problemi, specialmente nel nome del file `.c`. Un numero davanti aiuta a tenerli in ordine, per esempio `03_TerzoEsempio`. Poi *Finish*.

Le cartelle che compaiono a sinistra (*Header Files*, *Source Files*...) **non sono cartelle vere** sul disco: sono **cartelle virtuali** che l'**IDE** usa solo per organizzare il codice. IDE vuol dire *Integrated Development Environment*: un programma con tante funzioni per scrivere, compilare e provare il codice.

Nella cartella **Source Files**: pulsante destro → *New* → **`main.c`**. Mi raccomando **`main.c`** e non un altro tipo di file, perché cambia tutta la struttura. Il nome lo scegliete voi, ma **senza spazi** e senza caratteri speciali.

1.1 La struttura base del programma

Il programma ha sempre il blocco **`void main(void)`**: il termine tecnico è **funzione**. Tra le graffe mettiamo la parte operativa del programma.

Il **`main` non deve mai finire**: se arriva al `return` il programma si ferma, si "pianta". Non è un errore, è fatto così; ma la vostra scheda **non farà più niente** e bisognerà spegnerla e riaccenderla. Per garantire che non ci arrivi mai si usa un ciclo **`while`**, che ripete continuamente le stesse cose, con dentro una **condizione sempre vera**. Di solito scriviamo `while(1)`.

Si può anche fare come i programmatori "ad alto livello": definire due etichette **`TRUE`** e **`FALSE`** e scrivere `while(TRUE)`.

```
#define TRUE 1
#define FALSE 0
```

È una scelta elegante e va benissimo. Nei linguaggi ad alto livello `true` e `false` sono già definiti dalla libreria; qui **non esistono** e dobbiamo definirli noi: è una cosa in più da scrivere, ma non disturba. Dentro le parentesi del `while` basta qualunque cosa **sempre vera**: anche `3 != 7` andrebbe bene. Le etichette e le costanti (che non cambiano mai) si scrivono sempre in **MAIUSCOLO**: `TRUE`, non `true`.

1.2 I `#pragma` e `_XTAL_FREQ`

Come nella dispensa A04: menu **Window** → **Target Memory Views** → **Configuration Bits**. Nella finestra in basso impostate **FOSC = HS**, **WDTE = OFF** (il *watchdog*, che non vogliamo usare), **PWRTE, MCLRE, BOREN = ON**, **LVP, CPD, CP = OFF**: in breve "HS, off, 3 on, 3 off". Poi **Generate Source Code to Output** e copiate le **8 righe** `#pragma` in cima al codice. Salvatevele da qualche parte, così non dovete rifare tutta la procedura ogni volta.

Subito sotto definiamo anche **`_XTAL_FREQ`**, che serve per usare i ritardi. Il numero è la **frequenza del quarzo** montato: con un quarzo da 16 MHz si scrive 16 milioni, con uno da 8 MHz **8 milioni**. Nel simulatore lavoriamo a **8 MHz** (nello schema della McLab1 è disegnato un quarzo da 4 MHz, ma in PICSimLab la frequenza si sceglie nel campo **Clk (MHz)**: impostatelo a 8, **uguale** a `_XTAL_FREQ`): `8000000`, cioè l'8 seguito da **sei zeri** (tre zeri e altri tre zeri). L'errore più comune è uno **zero** in più o in meno: è la prima cosa da controllare quando "va più veloce" o "più lento" del previsto.

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config MCLRE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config CP = OFF

#define TRUE 1
#define FALSE 0
#define _XTAL_FREQ 8000000

#include <xc.h>

void main(void)
{
    //----- impostare la macchina (GPIO e altre periferiche)

    //-----
    while (TRUE)
    {
    }
    return;
}
```

Il programma ha **due parti**: prima del `while` si **imposta la macchina** (i **GPIO**, *General Purpose Input Output*, e più avanti altre periferiche: per ora solo PORTA e PORTB); dentro il `while` ci sono le **lavorazioni continue**.

2 La scheda del simulatore e i registri TRIS

Il simulatore **PICSimLab** ci dà una scheda (**McLab1**) già costruita da qualcun altro: non possiamo cambiarla, la prendiamo com'è. Il suo schema elettrico si apre dal menu **Help** → **Board**. Se fossimo noi a disegnarla potremmo scegliere che cosa far fare a ogni piedino; qui la scelta è **vincolata** dai collegamenti.

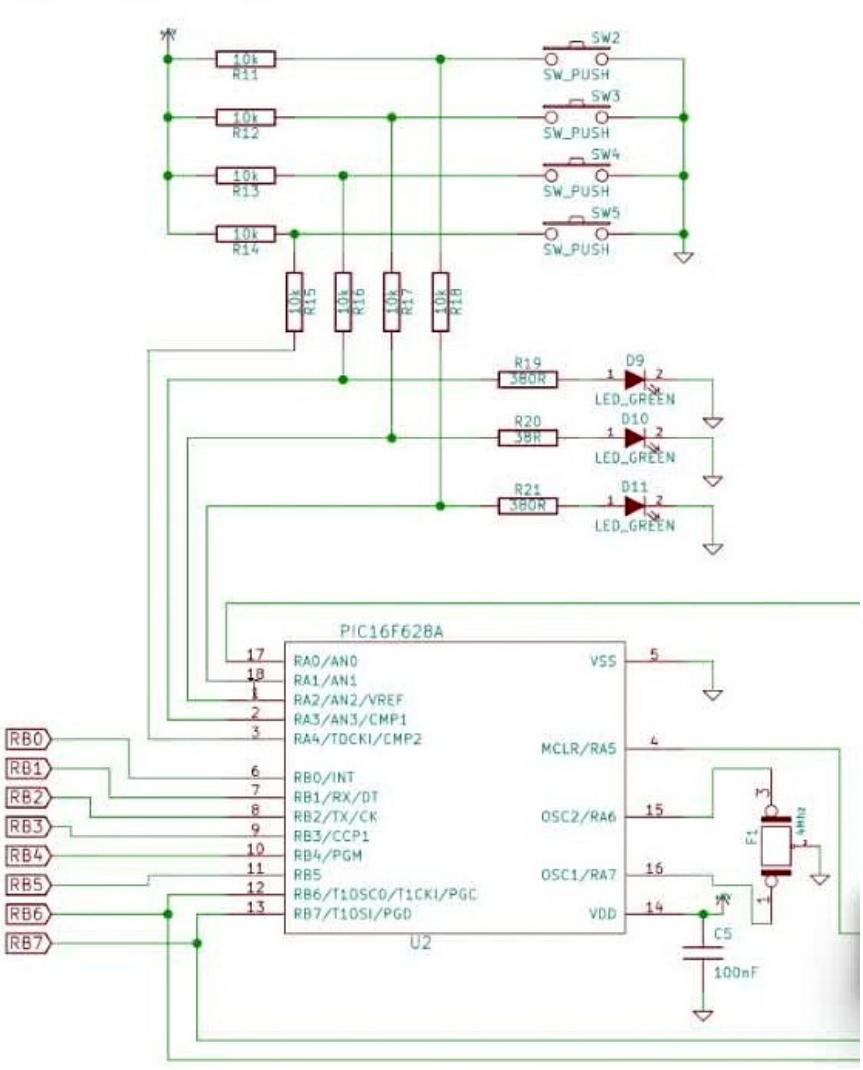


Figura 1: Parte dello schema della scheda McLab1 di PICSimLab: in alto i 4 pulsanti con i resistori di pull-up da 10 kΩ collegati a RA1–RA4, a destra in basso il transistor BC547 pilotato da RA0, a sinistra le uscite RB0–RB7 verso i LED.

- **Tutta la porta B** è collegata a dei **LED** (che siano LED singoli o segmenti di un display, sono pur sempre LED): quindi tutta la porta B è un'**uscita**, perché siamo noi a pilotare i LED.
- Sulla **porta A**, i piedini **RA1, RA2, RA3, RA4** sono collegati a dei **pulsanti con resistore di pull-up** (e anche a dei LED). Avendo già gli 8 LED della porta B, la porta A la usiamo **in ingresso**: RA1–RA4 sono **input**.
- **RA0** è collegato a un **BC547**, un **transistor**: una specie di **amplificatore di corrente**. Noi pilotiamo il transistor perché lui possa pilotare qualcosa che **consuma più corrente** (qui una lampada). RA0 quindi è un'**uscita**.
- RA5, RA6 e RA7 sono usati per l'**MCLR** e l'**oscillatore**: quello che scriviamo in quei bit è **irrilevante**. Mettiamo 0.

Nel registro **TRIS** un **1** vuol dire **ingresso** (*Input*), uno **0** vuol dire **uscita** (*Output*).

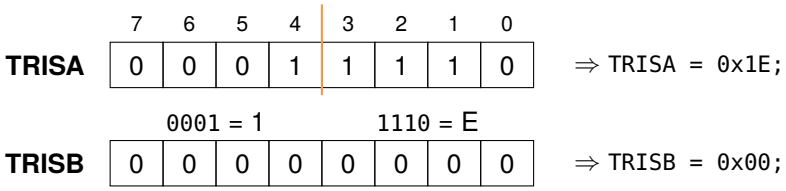


Figura 2: I due registri TRIS: in TRISA sono a 1 (ingresso) solo i bit 1, 2, 3, 4 (pulsanti); RA0 è un'uscita, i bit 5–7 non contano e valgono 0. In TRISB sono tutti a 0: tutta la porta B è in uscita. Spezzando gli 8 bit in due gruppi da 4 si legge subito il numero esadecimale.

```
TRISA = 0x1E;
TRISB = 0x00;
```

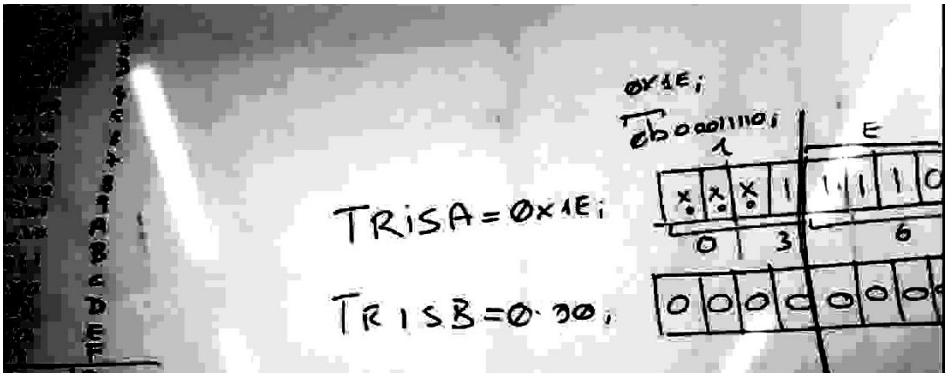


Figura 3: La lavagna: `TRISA = 0x1E` e `TRISB = 0x00`, con i bit di TRISA (le x sono i bit che non contano, messi a 0) e la stessa cifra scritta come `0b00011110`.

3 Contare in basi diverse

Per scrivere quei numeri ci sono più modi, e conviene capire **perché** si preferisce l'esadecimale.

3.1 Quante dita abbiamo?

Noi contiamo in **decimale** perché abbiamo **10 dita**: dieci simboli, 0 1 2 3 4 5 6 7 8 9. Finite le cifre, cominciamo a **combinarle**: 10, 11, 12… fino a 19; finite le combinazioni con l'1, passiamo al 2 (20, 21…), e avanti.

Esempio

Un pianeta con 4 dita per mano. Se fossimo nati su un pianeta dove si hanno **4 dita per mano**, cioè 8 in tutto, conteremmo in **ottale**: 0 1 2 3 4 5 6 7. Finite le cifre, si combinano: 10, 11, …, 17, poi 20, 21, 22… Il nostro numero 17 su quel pianeta si scriverebbe **21**: stessa quantità, scritta in un altro modo.
E con due dita in tutto? Avremmo solo **due simboli**, 0 e 1: è il **binario**. 0, 1, finito; si combina: 10, 11; finito: 100, 101, 110, 111; poi 1000…
E con 8 dita per mano? 16 dita in tutto, quindi **16 simboli**. I primi dieci sono uguali per tutti (0–9); per gli altri sei si usano le **lettere maiuscole**: **A**, **B**, **C**, **D**, **E**, **F**. Qui la F non è la F di un nome: è la cifra che vale **15**. Finiti i simboli, si combina: dopo F viene **10**, che vale sedici. È il sistema **esadecimale**.

3.2 La tabella di conversione

Questa tabella va **imparata**: con i primi 16 numeri in testa, la conversione tra binario ed esadecimale si fa a colpo d'occhio.

Dec	Ott	Bin	Hex	Dec	Ott	Bin	Hex
0	0	0000	0	11	13	1011	B
1	1	0001	1	12	14	1100	C
2	2	0010	2	13	15	1101	D
3	3	0011	3	14	16	1110	E
4	4	0100	4	15	17	1111	F
5	5	0101	5	16	20	1 0000	10
6	6	0110	6	17	21	1 0001	11
7	7	0111	7	18	22	1 0010	12
8	10	1000	8	19	23	1 0011	13
9	11	1001	9	20	24	1 0100	14
10	12	1010	A	30	36	1 1110	1E

3.3 Perché l'esadecimale

La conversione binario $\leftrightarrow$ esadecimale è semplicissima perché 16 è una **potenza di 2** (come lo sono 2 e 8). Basta **raggruppare** i bit **a quattro a quattro** (partendo da destra) e convertire ogni gruppo con la tabella: `0001 1110` $\rightarrow$ `1` e `E` $\rightarrow$ **`0x1E`**. Per l'ottale si raggruppano **a tre a tre**: `00 011 110` $\rightarrow$ 0, 3, 6 $\rightarrow$ **36** in ottale. In decimale lo stesso numero è **30**.

Per scrivere TRISA abbiamo quindi tre possibilità:

```
// decimale
TRISA = 30;
// binario
TRISA = 0b00011110;
// esadecimale
TRISA = 0x1E;
```

Ne basta una: sono tre modi di scrivere **lo stesso numero**.

- **Decimale**: per capire quali bit sono a 1 e quali a 0 (cioè quali piedini sono ingressi e quali uscite) bisogna fare **mille conti**. Scomodo.

- **Binario** (prefisso **`0b`**): si vede subito quali bit sono a 1. Con 8 bit sbagliare è possibile ma improbabile; ma su macchine più complesse, per esempio un **ESP32**, che usano parole da **32 bit**, scrivere 32 uni e zeri vuol dire sbagliare facilmente.
- **Esadecimale** (prefisso **`0x`**): **compatto** (poche cifre) e, una volta imparata la tabella, si legge **al volo**: vedo una E e so che è 1110. Per questo i **firmwaristi** preferiscono l'esadecimale.

Nota

E l'ottale in C? Anche l'ottale ha un prefisso: in C un numero che **comincia con uno zero** è scritto in ottale. `036` vale 30, come `0x1E`. Attenzione quindi a non mettere zeri "di abbellimento" davanti ai numeri decimali: `010` non vale dieci ma **otto**.

4 Accendere i LED della porta B

Dopo i TRIS possiamo far fare qualcosa alla macchina, nel `while`. Primo passo: accendere **solo RB0**. Nel registro **PORTB** il bit 0 corrisponde a RB0, il bit 1 a RB1 e così via: per accendere solo RB0 metto un **1 nel bit 0** e mi assicuro che **tutti gli altri** siano 0.

	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0
PORTB = 0x01	0	0	0	0	0	0	0	1

Figura 4: `PORTB = 0x01`: solo il bit 0 è a 1, quindi si accende solo RB0.

```
while (TRUE)
{
    PORTB = 0x01;
}
```

Scritto dentro il `while`, il programma continua a ciclare a velocità altissima ripetendo "accendi RB0" moltissime volte. Non è il modo più "pulito" di fare: è come dire a una persona **"stai seduto"**. Una volta detto, ci si aspetta che resti seduta finché non le si dice altro; continuare a ripeterle "stai seduto, stai seduto, stai seduto" è uno **spreco di risorse**. Per ora lo mettiamo nel `while` per comodità, perché poi ci servirà la parte che si ripete.

4.1 Compilare con un pulsante dedicato

Il pulsante con il **martello** in alto, premuto direttamente, esegue la **prima** voce del suo menu, che non è quella che ci serve; bisogna aprire la freccetta e scegliere la voce **in mezzo**. È scomodo, quindi conviene crearsi un pulsante apposta:

1. pulsante destro nella zona vuota della barra in alto → **Customize** (la voce in basso);
2. nella finestra che si apre: **New Toolbar**, e date un nome (per esempio il vostro);
3. la nuova barra compare in alto. Nell'elenco dei comandi scendete appena oltre la metà, trovate **Build Project** e **trascinatelo** (pulsante sinistro tenuto premuto) nella nuova barra;
4. chiudete: il pulsante è pronto.

Se la compilazione va a buon fine compare una scritta **verde** (`BUILD SUCCESSFUL`).

4.2 Caricare nel simulatore

Dalla terza riga dell'output, `Loading code from...`, copiate il percorso **fino all'ultima cartella `production`** (lasciando fuori il nome del file). In PICSimLab: **File** → **Load Hex**, incollate il percorso nella barra in alto e premete **Invio**, **non** "Apri": così la finestra ricarica il contenuto della cartella e vedete il file del vostro progetto (es. `03_TerzoEsempio`). Selezionatelo e aprite.

Dopo la prima volta non serve ricopiare il percorso: ogni volta che ricompilate (pulsante arancione), in PICSimLab basta **File** → **Reload**.

Attenzione

Non si accende niente? Controllate di aver scritto **`TRISB = 0x00;`**: senza quella riga la porta B resta in ingresso e i LED non si accendono, anche se `PORTB` è giusto. Gli altri errori visti ai posti:

- **`_XTAL_FREQ`** scritto in minuscolo, con due trattini bassi invece di uno, con uno spazio dentro, o **senza il numero** accanto: va scritto esattamente `#define _XTAL_FREQ 8000000`;
- **`#pragma`** sbagliati o mancanti (ne mancavano due righe): vanno rigenerati e ricopiati tutti e 8;
- il **punto e virgola** mancante a fine riga, una **graffa** girata al contrario o cancellata;
- **`0x00`** scritto con la lettera O invece dello zero.

Ascoltate le correzioni fatte agli altri: spesso il vostro errore è lo stesso del compagno precedente.

5 Far lampeggiare un LED

Per far lampeggiare RB0 lo **accendiamo** e poi lo **spegniamo**, con un **ritardo** in mezzo: senza ritardo il cambiamento sarebbe troppo veloce per vederlo. Il ritardo si scrive **`__delay_ms(500);`**: davanti a `delay` ci sono **due** trattini bassi.

```
while (TRUE)
{
    PORTB = 0x01;
    __delay_ms(500);
    PORTB = 0x00;
    __delay_ms(500);
}
```

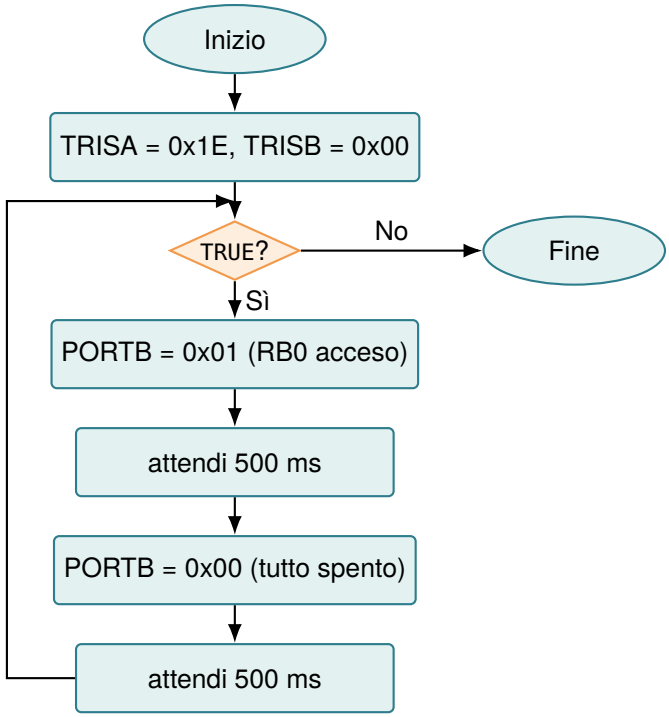


Figura 5: Lampeggio di RB0. La condizione `TRUE` è sempre vera: il ramo “No” e il nodo Fine non vengono mai raggiunti.

Per accendere **più LED** basta scegliere la combinazione giusta di uni e zeri. Uno sì e uno no: `0101 0101` → 5 e 5 → **`0x55`**. L’alternanza opposta: `1010 1010` → **`0xAA`**. Mettendo `PORTB = 0x55;` e `PORTB = 0xAA;` al posto di 0x01 e 0x00, i LED **pari e dispari si scambiano**. Con PORTB potete fare tutti i giochi di luci che volete.

6 La luce che scorre

Esercizio: **un solo LED acceso** alla volta, che **scorre** verso il basso: prima RB0, poi si spegne RB0 e si accende RB1, poi RB2, RB3… fino a RB7.

6.1 Quali numeri servono

Il bit a 1 si sposta di una posizione ogni volta. In esadecimale i numeri sono **1, 2, 4, 8** nella cifra di destra, e poi, quando l’1 passa nel gruppo di sinistra, **10, 20, 40, 80**:

LED acceso	binario	esadecimale
RB0	0000 0001	`0x01`
RB1	0000 0010	`0x02`
RB2	0000 0100	`0x04`
RB3	0000 1000	`0x08`
RB4	0001 0000	`0x10`
RB5	0010 0000	`0x20`
RB6	0100 0000	`0x40`
RB7	1000 0000	`0x80`

6.2 Prima versione: il “modo bovino”

Il modo più semplice è un bel **copia e incolla**: due righe (`PORTB = ...;` e `__delay_ms(500);`) ripetute **otto volte**, cambiando solo il numero.

```
while (TRUE)
{
    PORTB = 0x01;
    __delay_ms(500);
    PORTB = 0x02;
    __delay_ms(500);
    PORTB = 0x04;
    __delay_ms(500);
    PORTB = 0x08;
    __delay_ms(500);
    PORTB = 0x10;
    __delay_ms(500);
    PORTB = 0x20;
    __delay_ms(500);
    PORTB = 0x40;
    __delay_ms(500);
    PORTB = 0x80;
    __delay_ms(500);
}
```

Funziona, ma è il modo "**bovino**": poco furbo e **poco performante**. Se domani i passi fossero 200 o 400, dovrei scrivere 400 coppie di righe.

6.3 Seconda versione: un array e un `for`

Guardiamo il codice: ripetiamo **sempre gli stessi due comandi**, e l'unica cosa che cambia è il **numero** dentro. Quando qualcosa si ripete, in C si usa una **ripetizione**: `while` o `for`. Il `for` ha una variabile che **conta** i giri (0, 1, 2, . . . , 7): ci serve un modo per usare quel contatore per **pescare il numero giusto**.

Esempio

La classe numerata. Se invece di chiamarvi per nome do a ciascuno di voi un **numero di posizione** (0, 1, 2, . . .), e ognuno ha scritto su un cartello il suo "valore" (1, 2, 4, 8, 10, 20, 40, 80), posso dire "voglio il numero in posizione 3" e ottengo **8**; "in posizione 4" e ottengo **10**. Uso la **posizione** per recuperare il **valore vero**. Una struttura che contiene tutti i valori, ognuno pescabile con il suo numero di posizione, si chiama **array**.

Definizione

Array. Un **array** è un **contenitore di numeri** trattato come una variabile, ma con un **indice**: `leds[0]` è il primo elemento, `leds[1]` il secondo, e così via. Tutti gli elementi sono **dello stesso tipo**: o tutti `char`, o tutti `int`, o tutti `float`; non si possono mescolare. Si dichiara con il tipo, il nome, il **numero di elementi** tra parentesi **quadre** e i valori tra parentesi **graffe**, separati da virgole.

```
char leds[8] = {0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80};
```

Il passaggio logico è questo: al posto di `0x01` posso scrivere **`leds[0]`** ("dell'array `leds`, l'elemento di indice 0"); al posto di `0x02`, **`leds[1]`**; al posto di `0x04`, **`leds[2]`**, e così via. Adesso le otto coppie di righe sono **tutte uguali** tranne il numero tra le quadre, che vale 0, 1, 2, . . . , 7: proprio i valori del

contatore di un `for`. Possiamo chiamare il contatore come vogliamo: qui `conta` (di solito si usa `i`, che sta per *indice*). Questo è il primo dei **due modi** possibili, quello più vicino alla programmazione "ad alto livello" e più semplice da capire; l'altro, che lavora direttamente sui bit, lo vedremo più avanti.

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config MCLRE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config CP = OFF

#define TRUE 1
#define FALSE 0
#define _XTAL_FREQ 8000000

#include <xc.h>

char leds[8] = {0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80};

void main(void)
{
    //----- impostare la macchina (GPIO e altre periferiche)
    TRISA = 0x1E;
    TRISB = 0x00;

    //-----
    while (TRUE)
    {
        for (int conta = 0; conta < 8; conta++)
        {
            PORTB = leds[conta];
            __delay_ms(500);
        }
    }
    return;
}
```

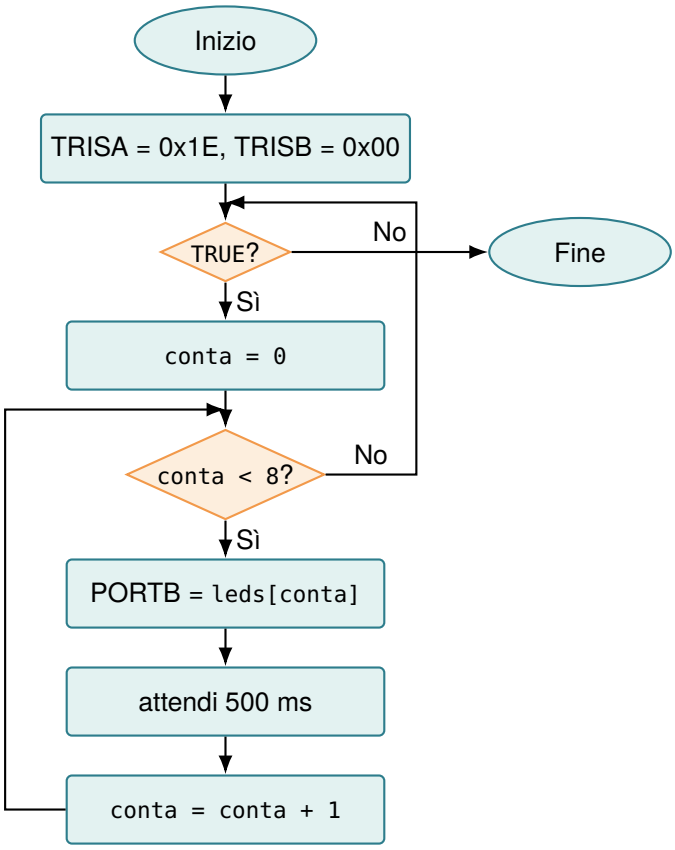


Figura 6: La luce che scorre: il `for` interno scorre l'array da 0 a 7; finiti gli 8 LED si torna al `while`, che ricomincia (il ramo "No" del `while` non viene mai raggiunto).

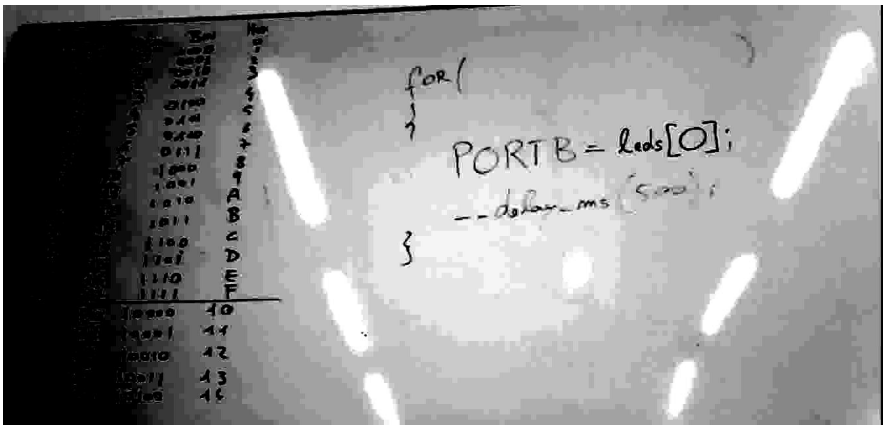


Figura 7: La lavagna a fine lezione: a sinistra la tabella decimale/ottale/binario/esadecimale costruita in classe, a destra il corpo del `for` con `PORTB = leds[0];` e `__delay_ms(500);`.

Attenzione

Gli errori più frequenti con gli array:

- i valori dell'array vanno tra **graffe** `{ }`, non tra parentesi tonde; l'indice va tra **quadre** `[ ]`;
- un **punto** al posto della virgola tra due valori (`0x01.`), o una virgola in più in fondo all'elenco;
- un **punto e virgola** dopo la parentesi del **`for`**: `for (...);` chiude il ciclo subito e il corpo viene eseguito una volta sola;
- una **graffa mancante** (quella che chiude il `for` o quella che chiude il `while`): seguite le linee verticali che MPLAB X disegna tra una graffa aperta e la sua chiusa;
- valori sbagliati nell'array (per esempio 3, 5, 6, 7 al posto di 0x08, 0x10...): devono essere **solo potenze di 2**, un bit alla volta.

Nella programmazione non si può sbagliare "di una virgola": il compilatore **non accetta** errori di questo tipo.

Nota

`char` o `int`? Se dichiarate l'array come `int leds[8]`, il programma funziona lo stesso, ma il compilatore dà un **warning** (un avviso, non un errore) quando scrivete `PORTB = leds[conta];`: PORTB è un registro da 8 bit e un `int` è più grande. Con **`char`** (8 bit, come il registro) l'avviso sparisce.

7 Cosa ricordare di questa lezione

- Nuovo progetto: **PIC16F628A**, **XC8**, nome **senza spazi** né caratteri speciali, file **`main.c`** in Source Files.
- Il **`main` non deve mai finire**: `while(1)` o `while(TRUE)` con `#define TRUE 1`. Costanti ed etichette in **MAIUSCOLO**.
- In testa: 8 **`#pragma`** (HS, off, 3 on, 3 off) e **`#define _XTAL_FREQ 8000000`** (quarzo da 8 MHz).
- **TRIS**: 1 = **ingresso**, 0 = **uscita**. Sulla McLab1: **`TRISA = 0x1E`** (pulsanti su RA1–RA4), **`TRISB = 0x00`** (LED).
- Basi: **decimale** (10 simboli), **ottale** (8), **binario** (2), **esadecimale** (16: 0–9 e A–F). Prefissi in C: **`0b`** binario, **`0x`** esadecimale, **`0`** iniziale ottale.
- Binario → esadecimale: **gruppi di 4 bit** (ottale: gruppi di 3). `0001 1110` = `0x1E` = 30 = 036.
- **`__delay_ms(500)`** con **due** trattini bassi. `0x55`/`0xAA`: LED alternati.
- Un **array** raccoglie valori **dello stesso tipo**, pescati con l'**indice** da 0: `leds[conta]` in un `for` sostituisce otto coppie di righe copiate.
- Compilare con **Build Project** (pulsante personalizzato), caricare con **Load Hex** (percorso fino a `production`, poi **Invio**), dopo con **File** → **Reload**.